

Типы данных

int - 123, -456
 str - "Привет, мир!"
 set - {1, 2, 3}
 None - n = None

float - 3.14, -0.001

list - [1, 2, 3]
 dict - {"name": "Анна",
 "age": 25}

complex - 2 + 3j
 tuple - (1, 2, 3)
 bool - flag = True

Для преобразования типов
 данных используются
 встроенные функции: **int()**,
float(), **str()**, **list()**, **tuple()**, **set()**,
dict()

Арифметические операторы

```
result = 5 + 3  # = 8
result = 5 - 3  # = 2
result = 5 * 3  # = 15
result = 5 / 2  # = 2.5
result = 5 // 2 # = 2
result = 5 % 2  # = 1
result = 2 ** 3 # = 8
```

Операторы присваивания

```
x = 5
x += 3  # x = 8
x -= 3  # x = 5
x *= 3  # x = 15
x /= 3  # x = 5
x //= 3 # x = 1
x = 5
x %= 3  # x = 2
x **= 2 # x = 4
```

Вложенные условия

```
x = 10
y = 5
if x > 5:
    if y > 2:
        print("Оба условия истинны")
```

Цикл while

```
count = 0
while count < 5:
    print(count)
    count += 1
```

Прерывание цикла с помощью break

```
count = 0
while True:
    if count >= 5:
        break
    print(count)
    count += 1
```

Пропуск итерации с помощью continue

```
count = 0
while count < 5:
    count += 1
    if count == 3:
        continue
    print(count)
```

Использование else в цикле while

```
count = 0
while count < 5:
    print(count)
    count += 1
else:
    print("Цикл завершен")
```

Функции**Объявление функции**

```
def greet(name):
    return f"Привет, {name}!"
```

Параметры функции

```
def add(a, b=0):
    return a + b
```

Лямбда-функции

```
double = lambda x: x * 2
```

Операторы сравнения

```
x = 5
y = 10
result = x == y  # = False
result = x != y  # = True
result = x < y   # = True
result = x > y   # = False
result = x <= y  # = True
result = x >= y  # = False
```

Операторы членства и идентичности

```
lst = [1, 2, 3]
result = 1 in lst  # True
result = 6 not in lst  # True
a = [1, 2, 3]
b = a
result = a is b  # True
a = [1, 2, 3]
b = [1, 2, 3]
result = a is not b  # True
```

Тернарный оператор

```
x = 10
result = True if x > 5 else False
print(result)
```

Цикл for**Базовая итерация**

```
for i in range(5):
    print(i)
```

Итерация по списку

```
lst = [10, 20, 30]
for num in lst:
    print(num)
```

Итерация по строке

```
s = "Python"
for char in s:
    print(char)
```

Использование else в цикле for

```
for i in range(3):
    print(i)
else:
    print("Цикл завершен")
```

Словари (dict)

```
d = {"key1": "value1"} # создать словарь
d["key1"] # вернуть значение по ключу
d.get("key1") # получить значение
d["key3"] = "value3" # + ключ-значение
d["key1"] = "new" # изменить значение
del d["key1"] # удалить элемент
d.pop("key1") # получить и удалить эл.
d.clear() # удалить всех элементов
d.keys() # получить все ключи
d.values() # получить все значения
d.items() # получить все ключ-значения
d.update({"key4": "value4"}) # обновить
словарь с другим словарем
```

Классы

```
class Person:
    def __init__(self, name):
        self.name = name
```

Методы класса

```
class Dog:
    def bark(self):
        return "Woof!"
```

Логические операторы

```
x = True
y = False
result = x and y  # = False
result = x or y   # = True
result = x not y   # = True
```

Оператор if

```
x = 10
if x > 5:
    print("x больше 5")
```

Оператор if...else

```
x = 4
if x % 2 == 0:
    print("x - четное число")
else:
    print("x - нечетное число")
```

Оператор if...elif...else

```
x = 0
if x > 0:
    print("x положительное")
elif x < 0:
    print("x отрицательное")
else:
    print("x равно нулю")
```

Использование логических операторов в условиях

```
x = 10
y = 20
if x > 5 and y > 15:
    print("Оба условия истинны")
```

Списки (list)**Создание списка:**

```
lst = [1, 2, 3, 4, 5]
Доступ к элементам:
first_element = lst[0]  # 1
last_element = lst[-1]  # 5
```

Методы:

```
lst.append(6) # добавить элемент в конец
lst.extend([7, 8, 9]) # добавить элементы
lst.insert(2, 10) # добавить 10 на позицию 2
lst.remove(3) # удалить 3 из списка
lst.pop(2) # удалить позицию 2 и получить
lst.clear() # удалить все элементы списка
lst.index(5) # вернуть индекс элемента
lst.count(2) # посчитать кол-во элемента
lst.sort() # отсортировать список
lst.reverse() # перевернуть список
lst.copy() # получить копию списка
```

Кортежи (tuple)

```
tup = (1, 2, 3, 4, 5) # создать кортеж
tup[0] # получить 0 элемент кортежа
tup[1:4] # сделать срез
```

Методы кортежей:

```
tup.count(2) # количество вхождений
tup.index(3) # получить индекс элемента
tuple(lst) # преобразовать список в кортеж
```

Множества (set)

```
s = {1, 2, 3, 4, 5} # создать множество
s.add(6) # добавить элемент
s.remove(3) # удалить элемент
s.discard(4) # удалить элемент /без ошибки
s.pop() # удалить и вернуть рандом. эл-т.
s.clear() # очистить множество
s1 | s2 # объединить множества
s1 & s2 # пересечение множеств
s1 - s2 # разность множеств
s1 ^ s2 # симметрическая разность
```

ООП

Наследование

```
class Animal:
    def __init__(self, name):
        self.name = name

    def make_sound(self):
        pass
```

```
class Dog(Animal):
    def make_sound(self):
        return "Woof!"
```

Инкапсуляция

```
class Car:
    def __init__(self):
        self.__speed = 0

    def set_speed(self, speed):
        self.__speed = speed
```

```
    def get_speed(self):
        return self.__speed
```

Полиморфизм

```
class Bird:
    def fly(self):
        return "Flying!"
```

```
class Fish:
    def swim(self):
        return "Swimming!"
```

```
def move(animal):
    if hasattr(animal, 'fly'):
        return animal.fly()
    elif hasattr(animal, 'swim'):
        return animal.swim()
```

Композиция

```
class Engine:
    def start(self):
        return "Engine started!"
```

```
class Car:
    def __init__(self):
        self.engine = Engine()

    def start(self):
        return self.engine.start()
```

Исключения

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Ошибка: {e}")

Базовые исключения:
ZeroDivisionError: Деление на ноль
TypeError: несоответствующего тип
ValueError: не правильное значение
IndexError: неверный индекс
Генерация исключений
raise ValueError
```

Встроенные функции

```
print() – вывод информации в консоль
len() – определение длины объекта
type() – тип объекта
input() – считывает ввод клавиатуры
abs() – абсолютное значение числа
round() – округление числа
range() – последовательность
sorted() – отсортировать список
sum() – сумма всех элементов
max() – макс число из списка
min() – мин число из списка
zip() – объединение элементов из
неск. последовательностей
enumerate() – перебор со значением и
индексом
reversed() – вернуть обратный
итератор
any() – True, если хотя бы один
элемент истинный
all() – True, если все истинные
```

Декоратор база

```
def декоратор(func):
    def обертка():
        # код перед вызовом функции
        func()
        # код после вызова функции
    return обертка
```

```
@декоратор
def привет():
    print("Привет!")
```

Пример декоратора

```
def debug_decorator(func):
    def wrapper(*args, **kwargs):
        print("")
        result = func(*args, **kwargs)
        print(f"")
        return result
    return wrapper
```

Генератор списков

```
lst = [x for x in range(10) if x % 2 == 0]
```

Генератор кортежей

```
tup = (x for x in range(10) if x % 2 == 0)
```

Функциональный генератор

```
def my_generator():
    for i in range(5):
        yield i
```

```
gen = my_generator()
```

Рекурсия

```
def factorial(n):
    if n == 0:
        return 1 # базовый случай
    else:
        return n * factorial(n - 1) # рекурсивный
случай
```